University of Waterloo
Faculty of Engineering
Department of Electrical & Computer Engineering

Low Power Hardware Cryptography

Group 2016.020

Carlos Moreno **Consultant**

July 2, 2015

This reports uses lots of very advanced knowledge. It scored 66.6 / 80. Section 3.2 is especially strong.

Contents

1 High Level Description

1.1 Motivation

Embedded devices are becoming increasingly common in society. Historically, security and cryptography in these devices has been either ineffective or entirely absent. The real constraints of power usage and responsiveness have outweighed the hypothetical constraint of security. However, as these devices gain Internet connectivity they gain exposure to an ever more hostile world.

HP has recently found that 70% of Internet-of-Things (IoT) devices connect to network services without authentication [1]. A consequence of this is that anyone connected to these insecure devices can very easily manipulate their data and functions. This can range from mildly annoying, in the case of a thermostat being controlled remotely from a malicious source, to dangerous, in the case of pacemakers being instructed to stop. There have even been attacks on insulin pumps where malicious requests were sent to give the patient the maximum dose regardless of actual need [2]. There is a legitimate need to be able to easily build security into embedded devices, and protect consumer IoT devices from being compromised.

1.2 Project Objective

The objective of this project is to enable low power embedded applications and devices to easily and efficiently use high quality cryptography, by adding a chip to their board that "just works". The power consumption should be lower, and the speed of encryption faster, than if the cryptography were implemented purely in software. The encryption should have a simple C application program interface (API) that is hard to use incorrectly, which embedded device designers can integrate with their projects. The device should be simple to integrate into both new and existing embedded projects.

Sometimes these objectives are at odds with each other. When they are, the security of the design must not be compromised as a result. A cryptographic module that doesn't correctly provide security is worse than no cryptographic module at all, as it provides a false sense of security where there is none.

1.3 Block Diagram

The block diagram in Figure 1 details the components of the embedded security solution.

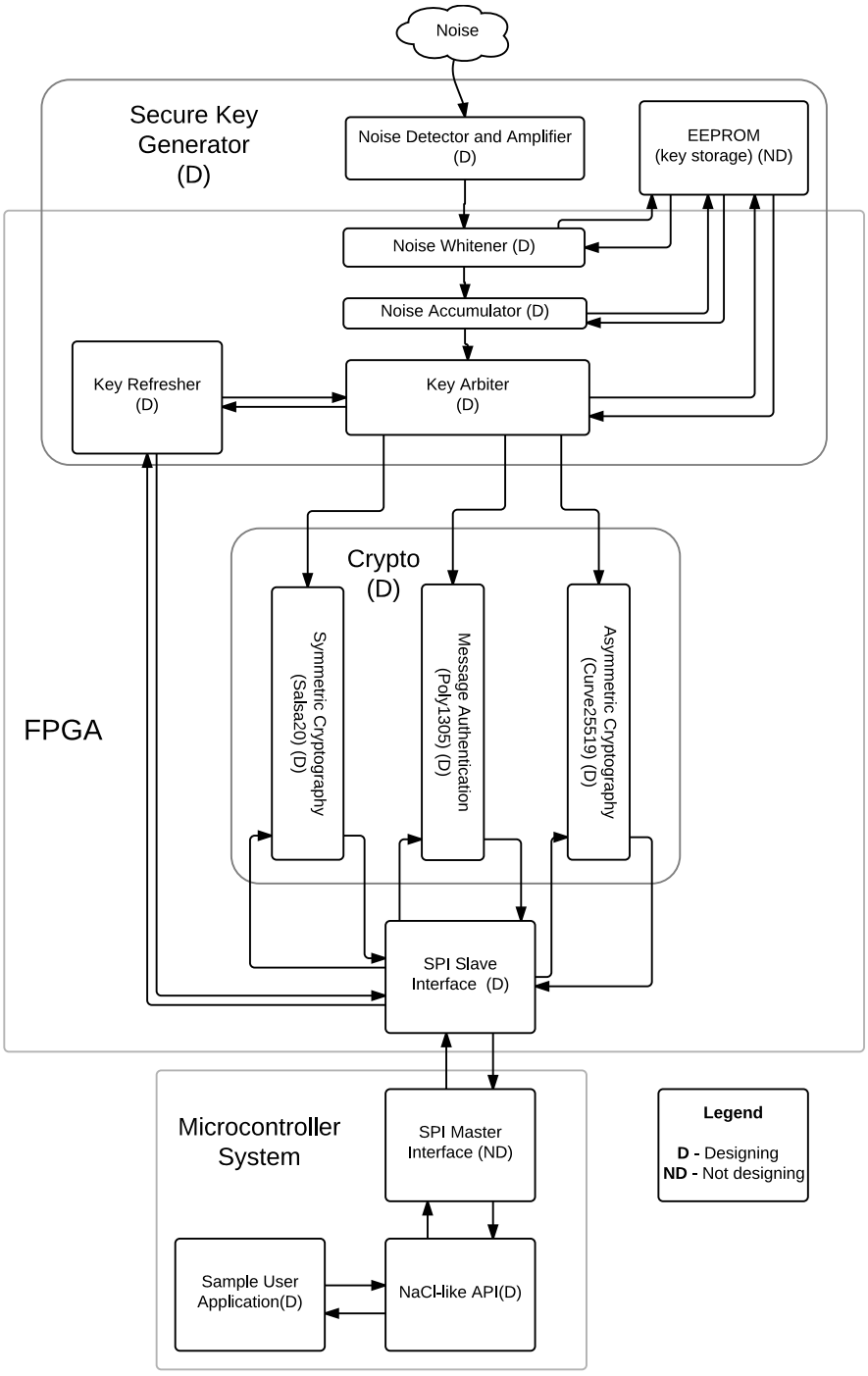


Figure 1: A block diagram showing the system's inputs, outputs, and abstract components.

1.3.1 Noise Detector and Amplifier

This module detects, amplifies and digitally encodes thermal noise. A purely software random number generator can only create pseudo-random numbers at best, because it is based off of an algorithm, and as such the numbers are predictable. By including random external noise, the filter can be carefully tuned to reduce correlation between generated numbers.

1.3.2 Noise Whitener (Rumba20 Instance)

Noise may be random, but it may also be biased. Another four instances of the Salsa20 block is used to construct a Rumba20 hash function. The block counter for the hash is stored in EEPROM.

1.3.3 Noise Accumulator

Noise is accumulated into a single 512-bit register. This register's output may be used to generate a new key at any time.

1.3.4 Key Arbiter

The key arbiter controls access to the private keys. Other subsystems request the keys from the key arbiter, which replies with a copy if they're authorized. This helps protect against key material leakage.

Note that the key material is never sent over the serial peripheral interface (SPI) bus, as those can be easily introspected with an oscilloscope.

1.3.5 Key Refresher

They Key Refresher provides an interface for the microcontroller to request a new private key, purging the old one. It responds to two different messages:

1. **Reset**, which generates a new keypair. The new public key is sent back.
2. **Pubkey**, which requests the chip's public key, such that other devices may send it secure messages.

The private key is never sent to the Key Refresher.

1.3.6 Message Authentication (Poly1305-Salsa20)

The Message Authentication block provides a hardware implementation of a variant of the Poly 1305-AES message authentication code (MAC) [3], but using Salsa20 instead of the advanced encryption standard (AES). It is a faster and more secure alternative to AES [4], and already implemented as the symmetric cryptography block.

1.3.7 Symmetric Cryptography (Salsa20)

The Symmetric Cryptography block provides a hardware implementation of Salsa20, a simple streaming, symmetric cipher [4].

1.3.8 Asymmetric Cryptography (Curve25519)

The Asymmetric Cryptography block provides a hardware implementation of the elliptic curve Curve25519 [5], used for Diffie-Hellman key exchange.

1.3.9 NaCl-like API

The networking and cryptography library (NaCl) provides a clear, hard to misuse C API to implement cryptographic functions in software [6]. The cryptographic hardware module will expose a C API aiming to replicate the NaCl API, providing a transparent transition for developers and applications from software to hardware cryptography.

1.3.10 SPI Slave Interface

All data exchange is done via a SPI interface. The master will provide data to encrypt or decrypt. Control commands will also be sent via the SPI bus. A custom framing format will need to be specified to allow for the security module to know which operations to perform. It is most likely that the FPGA will not have dedicated SPI hardware onboard, meaning an interfacing component will have to be designed to work with the GPIO (general purpose input/output) pins on the board.

2 Project Specifications

Table 1 lists the functional specifications of the device: what it does. They are classified as essential and non-essential, where if an essential specification is not met the design is considered unsatisfactory.

Table 2 lists the non-functional specifications of the device: how it works. They are similarly classified as either essential or non-essential.

Table 1: Functional Specifications

Subsystem	**Specification**	**Additional Details**	**Necessity**
Secure Key Generator	This subsystem must be cryptographically secure.	In order to make sure the keys generated by the module are truly random and do not follow any sort of pattern, the keys generated is subject to the Discrete Fourier Transform (Spectral) Test as outlined by the National Institute of Standards and Technology. A p-value is a measurement of how random a random number generator is. A value of 0 is completely non-random, and 1 is perfectly random. The p-value for the resulting from the test for the secure key generator must be greater than 0.1. [7]	Essential
Symmetric Cryptography Block (Salsa20)	The Salsa20 block must be able to encrypt and decrypt data faster than a software implementation of AES on an ARM Cortex M0 processor.	Salsa20 is a symmetric cipher that accomplishes goals similar to AES. AES implementations on Cortex-M0 have a throughput of 200 kB/s [8] [9].	Essential
Power Consumption	The power consumption of the prototype must be smaller than a Cortex-M0 for its software equivalents.	Unfortunately, no consistent studies currently exist to power profile the power usage of these cryptographic algorithms on ARM Cortex-M0 microcontrollers. As such, the group will have to perform this study to set an accurate benchmark for reasonable power performance. The benchmark should provide the power performance of SHA256, AES-128, Curve25519 and Poly-1305 on an ARM Cortex-M0 microcontroller. The device must be beat the power performance found in this benchmark.	Essential

Table 2: Non-Functional Specifications

Sub-system	**Specification**	**Additional Details**	**Necessity**
All	The device must be physically secure.	An attacker with physical access to the device should not be able to compromise the secret keys.	Essential
Secure Key Generator	It must be impossible for the central processing unit (CPU) to gain access to any secret cryptographic keys.	All cryptographic functions are performed on-device. Therefore, it should be possible to "air gap" the link between the CPU and the cryptography chip. This ensures that private keys are safe even if the CPU is compromised.	Essential
SPI Bus	The SPI bus clock line must be able to run at 100 kHz, 400 kHz, 1 MHz, 2 MHz and 5 MHz.	These varying speed settings are to ensure that almost any microcontroller capable of connecting to the Internet is able to use the module.	Essential
Secure Key Generator	The system must generate a public/private key pair in less than 1000 ms.	If this is impossible without compromising security, it may take longer. This time is based off the time it takes for a Cortex-M0 to generate a key pair[10].	Non-Essential
Cryptographic Hardware Logic	The area of the FPGA used by the implementation of the cryptographic algorithms must be less than 22,000 logic elements.	This is the size of the Altera Cyclone IV, the device planned to be used. It is a device sized specifically for use in low-power, low-cost applications. The design should fit on this device.	Non-Essential
API	The API of the hardware must be identical to that of the Networking and Cryptography Library [6]	Having an identical API to that used in existing software solutions will make porting of existing designs trivial for users. NaCl has also been explicitly designed to be "hard to misuse", which is an extremely valuable property for a cryptography library.	Non-Essential

3 Detailed Design

In each section of the detailed design, hardware dataflow diagrams are presented for FPGA components, and schematics for external components.

3.1 Secure Key Generation

3.1.1 Key Arbitration and Storage

In order to properly provide each cryptographic module its necessary keys, a key arbiter is made to initiate key generation and transfer between the components. The keys are stored in a tamper-free EEPROM to avoid physical inspection of secret data.

3.1.2 Noise Generation

This subsystem has 3 main components to it. The first is the source of noise. Any component in an electronic circuit can generate some amount of thermal noise, though these are often generated by using either transistors or diodes. The component chosen to generate the noise in this design is a Zener diode. A Zener diode in reverse bias can generate uniformly distributed noise from 0-10 MHz[11]. Noise of this frequency allows the subsystem to generate 10 million bits of entropy per second, allowing for the parameterization of the Rumba20 noise-whitener in 0.0001536 seconds, clearly making the bottleneck of the key generation subsystem the noise whitening and not the generation.

The second component is the amplification of the noise. The six stage amplifier amplifies signals to 2 million times their original amplitude, which can take a 1.0 μV signal and change it to a 2.0 V signal. The LTC6244 is a dual 50MHz, low noise, rail-to-rail, complementary metal-oxide-semiconductor (CMOS) operational amplifier. The specifications call for at least 1 MHz gain-bandwidth product To be able to operate at high speeds and provide enough gain. The op amp needs to be able to run on the 3.3 V available from the FPGA rail, and be able to detect microvoltages from the noise generated in the diode. These parameters, in addition to simulating to other similar op amps for comparison, made the LTC6244 the best option for this purpose. Due to the fact that the rail to rail voltage is entirely positive, this introduces a DC bias into each amplification stage. As a result, different iterations of capacitors and voltage dividers have been introduced between each stage, as described in Figure 2, to bring the smallest signal value back down to 0 V.

The third is the comparator stage. A high speed op amp takes in the amplified signal, and compares it to a centered voltage value. If the input is greater than the centered voltage, the output is the value of the V+ voltage, which is attached to the rail voltage of 3.3 V; if the input is less than the centered voltage, the output is equal to the V- value of the op amp, which is connected to ground to give a value of 0 V. This process is completed to change the original amplified random noise into digital values of entropy that can read on the FPGA via the on-board general purpose input/output pins (GPIOs) for noise whitening.

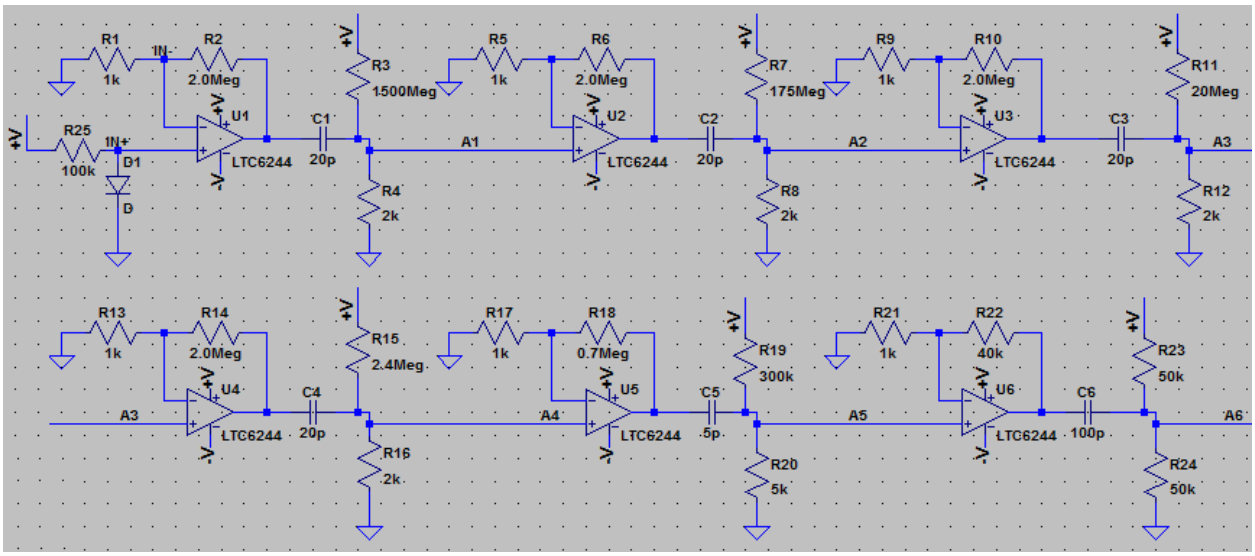


Figure 2: Noise Generation and Amplification

3.1.3 Noise Whitening (Rumba20)

Rumba20 [12] compresses 1,536 bits of entropy down into a whitened 512 bits with collision resistance. These bits can be accumulated over time with feedback, resulting in large changes to the random number state as new entropy comes in. Note that the core of Rumba20 is just a carefully parameterized HSalsa20, which is described in more detail in [4], as well as the Symmetric Encryption section of this paper.

Figure 3 describes a full Rumba20 block.

Note that in terms of raw hardware requirements, only a single HSalsa20 block needs to be synthesized, as all 4 stages of the Rumba20 may be performed by the same hardware.

This component satisfies, in part, the cryptographic security of the Secure Key Generator subsystem. If the Rumba20 hash construction is secure, then the noise whitener will meet performance criteria. It is also fast enough to satisfy the design constraint of how quickly secure key can be generated, as this design has latency of 496 clock cycles, which is equivalent to a throughput of 4.96 μs per 512 bits of whitened entropy, assuming a very conservative clock frequency of 100 MHz. This is more than enough to satisfy the criteria of 256 bits of entropy per second.

Two other designs were considered for whitening noise: arithmetic coding and SHA-256.

Arithmetic coding uses a window of the last few bits of entropy to reduce redundancy, similar to Huffman Coding. This is more or less isomorphic to whitening the input, since any biases will lead to redundant bits. This method was rejected because very little is known of its security properties. If someone can influence the noise source in a specific way (for some as-yet-unresearched definition of specific), this scheme might be broken.

The other possible design was a whitener based on SHA-256, in a chaining-block-cipher construction. This is a much more secure way of whitening the entropy, and is extensively studied as a cryptographically secure construction. However, the trade-off is that yet another cryptographic primitive must be implemented in hardware, and SHA-256 isn't especially simple. Additionally, errors in its design are hard to detect, since a poor hash function still looks like a bunch of random numbers even if they're insecure.

Finally, the Rumba20 design was chosen. It can use, at its core, Salsa20. This is the exact same hardware as is required for the symmetric encryption, and therefore are being extensively tested. It also has a security proof [12], which defines Rumba20's invertibility as a function of how good of a block cipher Salsa20 is. This is much more comfortable than arithmetic coding, and much easier to implement than SHA-256. Therefore,

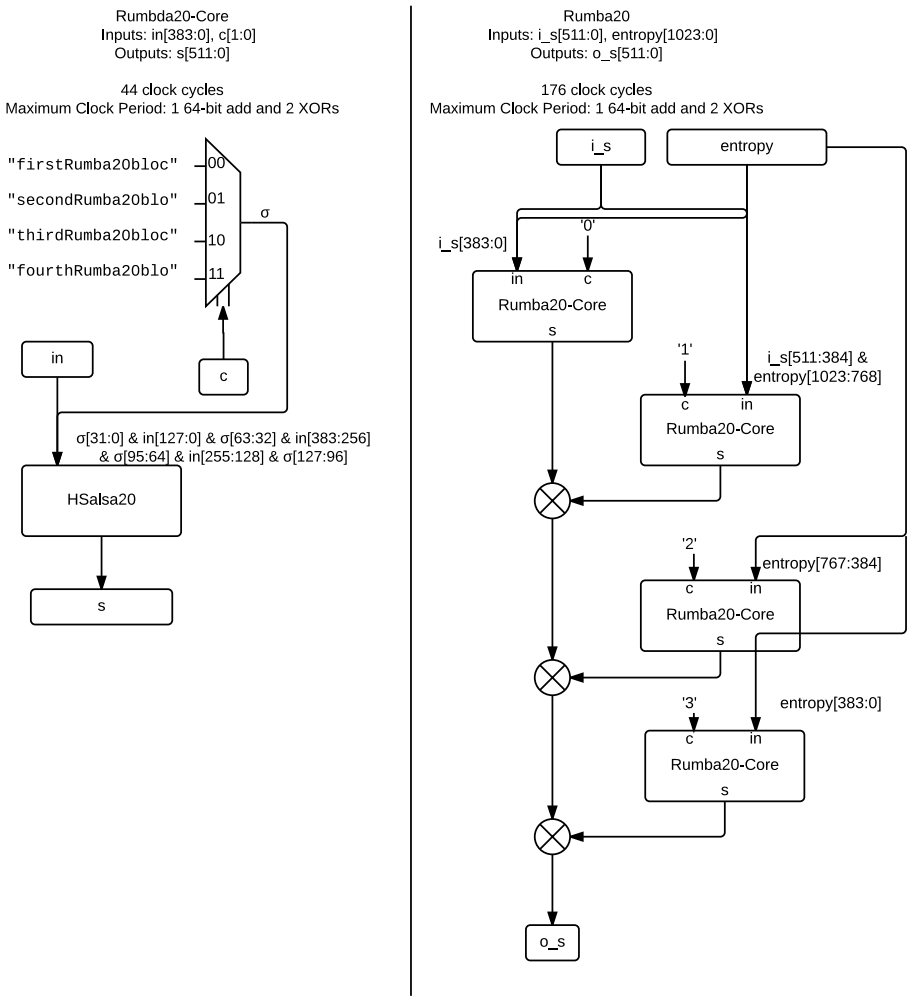


Figure 3: Rumba20

it was chosen.

There are a variety of configurations Rumba20 may take, mostly based on how the Salsa20 core is parameterized. It can either 1, 2, or 4, or 8 cycles per round, for 60 total rounds of hashing. The amount of hardware necessary to implement it is inversely proportional to the cycles per round. At one extreme, there's the 1 cycle per round, which requires 16 eight-round functional units and will cause Rumba20 to take 72 clock cycles to complete. At the other extreme, there's 8 cycles per round, which requires 8 eight-round functional units and will cause Rumba20 to take 496 clock cycles per block. Since key generation does not lie on the cryptographic critical path (since it only needs to happen once per connection), minimizing hardware is prioritized. Therefore, Salsa20 is configured to take 8 clock cycles per round.

3.1.4 Noise Accumulation

Noise is accumulated into a single register. Every time 1,024 bits of entropy are generated, they, along with the previous 512 bits accumulated, are used to parameterize Rumba20. The result of the Rumba is the new

result of the noise accumulation. This is made especially easy by Rumba20's construction, which has 1,536 bits of input and 512 bits of output, implying an obvious feedback construction with 1,024 bits of new input every iteration.

When a random number is requested from this module, the accumulator will block until at least 1,024 bits of real entropy have been whitened. This ensures no random bits are sent out twice, which would be a huge security hole. Only the first 256 bits output from Rumba20 are returned, since that's how many are needed for generating a single Curve25519 key pair.

This subsystem helps satisfy the secure key generation design criteria. By accumulating noise, the current random bits are a function of all entropy seen since the device was powered on. This will help defend against short-lived attacks. Even if an attacker compromises the entropy source, there are still 256-bits of secret entropy from before the compromise to foil attacks. This is more than enough bits to stop bruteforce attacks.

Another design considered was to run Rumba20 with 1,536 new bits of entropy, and XOR its result into a 512-bit register every iteration. This was a very ad-hoc construction, and not at all secure. If an attacker compromises the entropy source for a short enough time to get it to output 256 known bits, they can observe the entropy in the future by XORing again. Therefore, this was rejected for being an insecure construction.

The construction used (with feedback of 512 bits) is the one recommended in the original Rumba20 paper [12]. Only using 256 bits of output was not in the paper, but is a convenient, novel design decision. It adds defense in depth to the entropy source, which is a very commonly attacked component.

Quantitatively, the design of this subsystem has no hardware cost. Hooking up Rumba20 in feedback can be done purely through routing, so no additional hardware needs to be synthesized.

3.2 Cryptography

The cryptographic algorithms being implemented have all been published in the public domain and been proven cryptographically secure. The analysis and design in this report focuses on the design of hardware implementations of the algorithms, such that the timing and area requirements of both the FPGA and of the project specifications are met. The security of cryptographic algorithms is mathematical in nature, and thus to be robust and secure requires working with large numbers on the order of 128 to 1024 bits or more. Working with numbers of this size imposes restrictions on the hardware design. The following table displays maximum clock frequencies for add and multiply operations measured using timing analysis on the selected FPGA.

Table 3: Timing analysis results of mathematical operations on the Cyclone IV FPGA

Operand size (bits)	Operation	Maximum Clock Frequency (MHz)
64	Addition	304
128	Addition	119
256	Addition	63
128	Multiplication	45

Performing large additions and multiplications is too costly as it reduces the clock frequency to far below the maximum memory speed of 143 MHz, and even lower than the micro-controller's system clock speed of 50 MHz. Additionally an area analysis revealed that just one 128 bit multiplier uses 49% of the chip's available interconnects, making the remainder of the design unfeasible. For this reason, in depth analysis was performed on how to efficiently implement the cryptographic algorithms using pipelined functional units to reduce their area, increase the clock speed, and allow for efficient reuse of hardware. Cryptographic algorithms rely on the modulo operation, which is not synthesizable in hardware, and typically must be replaced with a division, a multiplication, and a subtraction. A design analysis was performed on developing

large modular multipliers in hardware, as they are required in the design of each cryptographic component, and lie on their critical paths.

3.2.1 Modular Arithmetic Requirements

Both message authentication and asymmetric encryption rely heavily on modular arithmetic cores. This section covers the design of those primitives.

In order to successfully perform Curve25519 point multiplication, the following operations must be supported: $(A+B) \bmod (2^{255}-19)$, $(A-B) \bmod (2^{255}-19)$, $(A*B) \bmod (2^{255}-19)$, and $A^{-1} \bmod (2^{255}-19)$.

In order to successfully construct Poly1305 message authentication codes, the following operation must be supported: $AB \bmod (2^{130}-5)$.

3.2.2 Modular Addition of an 8-bit Number

A key support to successfully performing modular addition is the ability to add a small 8-bit number (ϵ) to a large 255-bit number (A), returning the fully reduced result $A + \epsilon \bmod (2^{255} - 19)$. The design for this functional unit is shown in Figure 4. The key idea behind this unit is that the large 255-bit A is split up

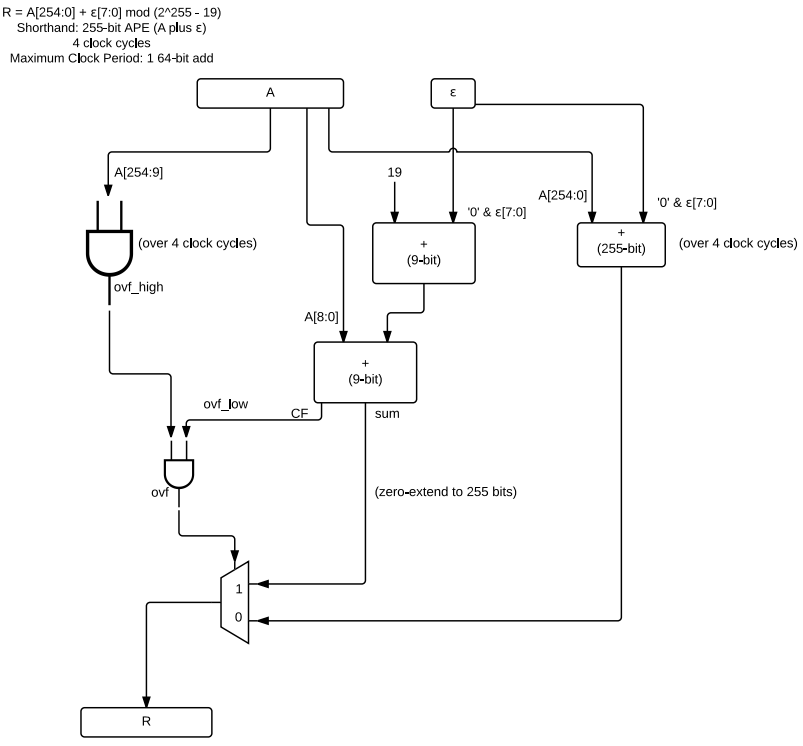


Figure 4: Addition of an 8-bit word to a 254-bit number.

into two parts: the low bits and the high bits. A modular overflow is only possible if all the high bits are set and $\epsilon + 19$, when added to the low bits, generates a carry. If both these conditions are true, then the amount overflowed is used as the return value. Otherwise, the natural sum is returned.

By splitting up the problem in two, the AND gate chain can run in parallel with the 255-bit add, over 4 clock cycles. This reduces the clock period requirements of this functional component to a single 64-bit add, which is known to be able to run at 304 MHz.

Additionally, although this unit is explicitly designed for Curve25519, it is trivial to extend to Poly1305 simply by replacing constants, and reducing register sizes. It also conveniently doesn't required reduced inputs. Any 255-bit number, and any 8-bit number can be added together to generate their proper sum, $(A + \epsilon) \bmod (2^{255} - 19)$.

No alternative designs for this component were explored, since it is only necessary for Poly1305 modular multiplication and Curve25519 point addition, which did have alternative designs.

The design has been formally verified to be equivalent to the naive circuit for addition of an 8-bit word, using an SMT (Satisfiability Modulo Theories) solver.

3.2.3 Modular Adder

The unit to do modular addition of an 8-bit number can be used to easily build a full blown modular adder. The details of this expansion is outlined in Figure 6. However, the key insight is that a normal sum may be calculated, and then through a bit of algebra:

$$\begin{aligned} & (2^{256} + x) \bmod (2^{255} - 19) \\ = & (2(2^{255} - 19 + 19) + x) \bmod (2^{255} - 19) \\ = & (2(0 + 19) + x) \bmod (2^{255} - 19) \\ = & (2 * 19 + x) \bmod (2^{255} - 19) \end{aligned}$$

And since $2 * 19 < 2^8 - 1$, the result of the modular add can be reduced to an addition of $\epsilon = 2 * 19$, using the existing addition of an 8-bit number unit.

Overall, this operation takes 8 clock cycles, and has a maximum clock frequency restriction of, again, a single 64-bit adder: 304 MHz.

Once again, this design is applicable to both Curve25519 and Poly1305. Both curves have similar construction, and are only different in the particular constants they use, and their bit width.

The alternative considered is a naive approach of adding comparing and subtracting, as shown by the pseudo-code in Figure 6. Unfortunately, this design does not allow for any parallelism and the clock frequency is restricted to that of a 256-bit add as described in Table 3. Doing the 256-bit adds over 4 64-bit adds is another considered design, however the lack of parallelism of the modulo operation still hindered the design's performance. Therefore this alternative was iterated upon to to increase parallelism and reduce clock frequency to come up with the final design.

```
uint256_t mod_add(uint256_t a, uint256_t b, uint256_t p){
        //returns (a + b) % p
        uint256_t sum = a + b;
        if (sum > p){
                return sum - p
        }
        else {
                return sum;
        }
}
```

Figure 5: Naive adder pseudocode

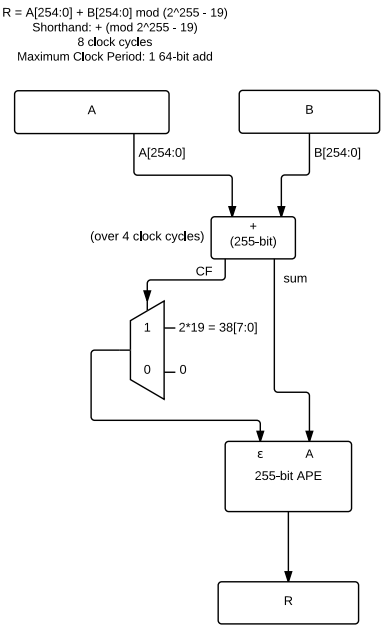


Figure 6: Modular Adder Dataflow

3.2.4 Modular Subtractor

Figure 7 contains the detailed design for performing modular subtraction. Unlike the adder, this relies on inputs that are already reduced (i.e. between 0 and $2^{255} - 18$, inclusive). With this restriction, subtraction is easy. Do a traditional subtraction, then add $2^{255} - 19$ if result is negative. The downside of this approach is that it takes a long time. 256-bit adds don't come cheap, so this unit has an overall cost of 8 clock cycles. However, the core of a subtractor is the same as an adder, so it is essentially just a single 64-bit adder, 64 inverters, and some control circuitry.

Once again, the clock period restriction of this functional unit is 304 MHz, for the same reason as the adder units: the largest-latency component is a single 64-bit adder.

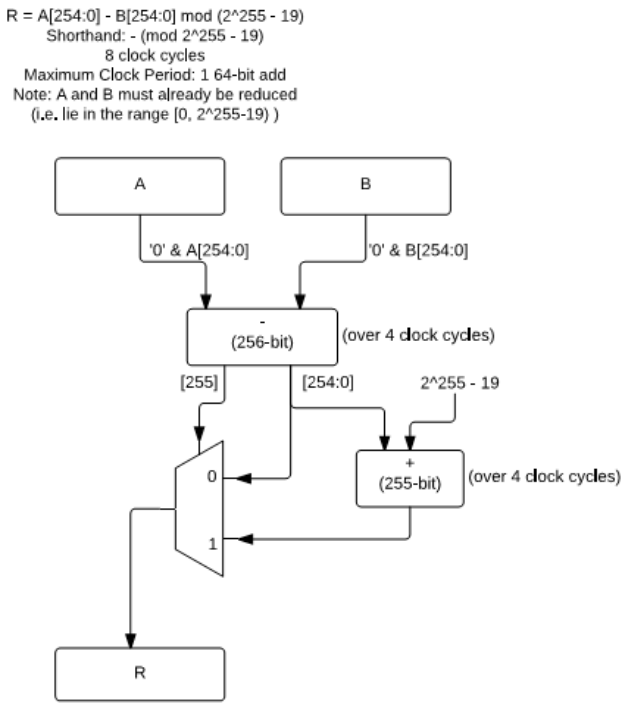


Figure 7: Modular subtraction.

3.2.5 Modular Multiplier

Modular multiplication is performed in three steps: building a look up table, a carry-save-addition (CSA) loop, then building the final sum. The core algorithm for this operation is detailed in [13].

The first step is detailed in Figure 8, while the other two are in Figure 9. This method is based off of the work done by David Amanor in his Master's Thesis[13]. However, the thesis neglected details on how to build the look up table and perform final addition efficiently. These are unique contributions to the field.

To build the lookup table (LUT), a unit performing the addition of a small constant clocks in the right hand side of the multiplication, with the constant being a given even multiple of 19, depending on which LUT entry is targeted. Even LUT entries are simply constants.

The process of building the initial look-up table takes 16 clock cycles: 4 clock cycles per addition, and there are 4 additions. It also requires 1.02 kb of RAM to store the entries. Since Cyclone IV RAM is allocated in 1 kB blocks, a full block must be used resulting in a total RAM usage of 1 kB.

The CSA loop runs for 255 clock cycles, performing a single carry-save addition per clock cycle. The bits of the left hand side of the multiplication are barrel-shifted in one bit at a time, driving the LUT lookups.

Finally, when the barrel-shifter completes, the output of the carry-save adder (two 255-bit integers) are used as inputs to the modular addition unit, to construct the final result. This stage takes 8 clock cycles.

Overall, a single modular multiplier has a maximum clock period of 304 MHz (same critical path for all modular math units: 64-bit adders), and takes $12 + 255 + 8 = 275$ clock cycles.

This design is also portable to Poly1305, since only constants need to be adjusted.

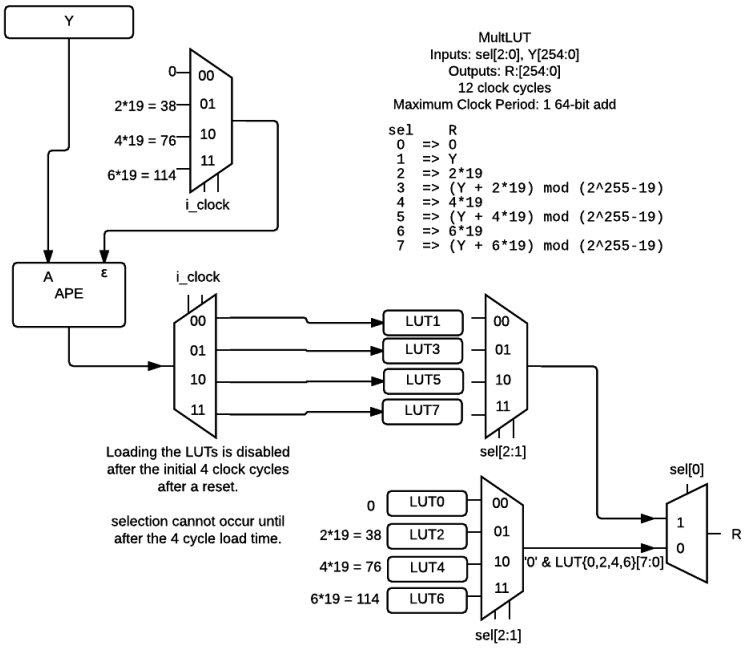


Figure 8: Modular multiplication LUT construction.

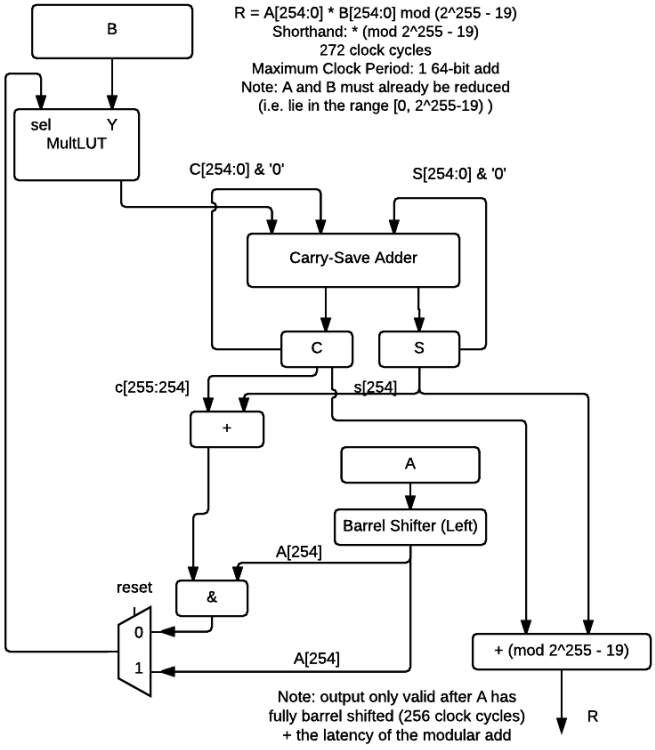


Figure 9: Modular multiplication.

3.2.6 Modular Inversion

Since $2^{255} - 19$ is prime [5], Euler's Theorem may be used to simplify finding the multiplicative inverse [14]. Simply, it states that:

$$A^{-1} \equiv A^{(2^{255}-19)-2} \bmod (2^{255} - 19)$$

By performing the exponentiation via iterated squaring and multiplying, the inverse may be computed efficiently using the modular multiplier component connected in feedback. This follows more or less the algorithm demonstrated for exponentiation by squaring in [15]. It requires two parallel modular multiplier units, and requires 32 multiplications (with 2 multiplications running in parallel at all times). Therefore, this has an overall latency of $275 * 16 = 4,400$ clock cycles. This is a small fraction of the overall time necessary to perform scalar multiplication (around 10% in software implementations), but does lie on the critical path.

This method has direct applications to Curve25519 point addition. In the Montgomery ladder (described in Section 3.2.7), the final stage is finding the multiplicative inverse of a number $\bmod 2^{255} - 19$. This method is key to efficient asymmetric encryption, an essential requirement of this device.

Another possible design would be to use the Extended Euclidean Algorithm. This method has the potential to be quicker, but is more complicated to implement in hardware. Additionally, it takes a variable amount of time depending on the input, something difficult to implement in hardware. This also makes it subject to timing attacks and power analysis attacks. Therefore, this option was rejected.

3.2.7 Asymmetric Encryption (Curve25519)

Public key and shared key generation in Curve25519 both rely on the multiplication of some scalar by some point on the elliptic curve. To generate a public key, a randomly chosen 32 byte private key (scalar) is multiplied by the Curve25519 base point which is the public 32 byte string `{0x9, 0x0, ..., 0x0}`. To generate the shared secret, parties Alice and Bob can multiply each others public key (a point on the curve) by their own private key.

The design for the scalar multiplier was based on the well optimized software designs for scalar multiplication within the NaCl Curve25519 implementation. This implementation relies on the use of the Montgomery Ladder to perform a "double-and-add" operation as well the double-and-add algorithm to perform point multiplication. The main alternative to the Montgomery Ladder is the naive double-and-add algorithm which performs point multiplication with fewer operations but not in a fixed amount of time. Unfortunately, this boost in performance with the naive double-and-add algorithm sacrifices resistance to side-channel attacks which provides attackers with an easy medium for both timing and power analysis attacks [16]. For this reason it was excluded as a possible design alternative. Given that the Montgomery ladder performs these operations in hardware with hardware optimized modular operations, it is predicted that these operations should use less power and perform computations faster than the Cortex M0 which satisfies are performance constraints. This cannot be proven directly until a prototype is built but research shows that dedicated cryptographic hardware tends to satisfy this [17].

The Montgomery Ladder for Curve25519 provides an efficient way to double a point and add a scalar to a point. D. J. Bernstein's Curve25519 paper also provides a data-flow diagram of the Curve25519 Montgomery Ladder [5]. This data-flow diagram was easy to translate into hardware. Multiple iterations on the design of the hardware Montgomery Ladder were done to increase parallelism within clock cycles and reuse functional units.

Due to the complexity of the modular multiplier, it is predicted that the area of the modular multiplier is too large to include more than one in the design. For this reason, there are no pipeline stages or parallel multiplies in the design. Once the modular multiplier is designed in Verilog, if the area is small enough to permit a second modular multiplier, this is being considered in order to improve the throughput of

Curve25519. The block diagram of the Montgomery ladder is shown in Figure 11 and the design of the Montgomery Ladder is shown in Figure 10.

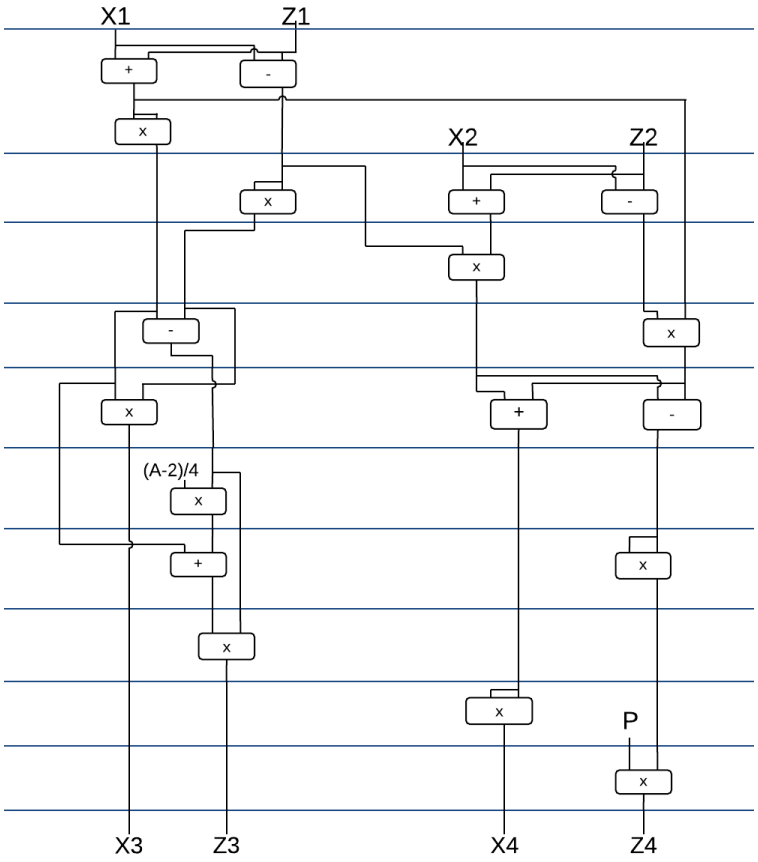


Figure 10: Curve25519 Montgomery Ladder

Scalar multiplication is achieved by performing double-and-add operations using the Montgomery ladder. The algorithm chosen was specifically designed for use with the Montgomery Ladder. A total of 255 iterations are done, one for each bit of the scalar multiplicand. The algorithm used for scalar multiplication is the same one used for the NaCl implementation of Curve25519 as it is both very efficient and protects against side channel attacks [16]. The scalar multiplier relies on a function called `CSwap` which swaps the X inputs and Z inputs to the Montgomery ladder if some input bit, C, is equal to 1. The design of this in hardware is shown in Figure 12.

The scalar multiplier is almost entirely a function of the Montgomery Ladder. The CSwap and xor functions done in each iteration of the scalar multiply are negligible in cost of time, area and power compared to the Montgomery ladder. The design chosen also does the scalar multiply in a fixed amoutn of time which protects from timing and power analysis attacks. The design of the scalar multiplier could be made to use less area and perform its computation in fewer operations, but this requires using a double-and-add method other than the Montgomery Ladder which leaves the device open to side-channel attacks as discussed earlier. The design of the scalar multiplier is shown in Figure 13.

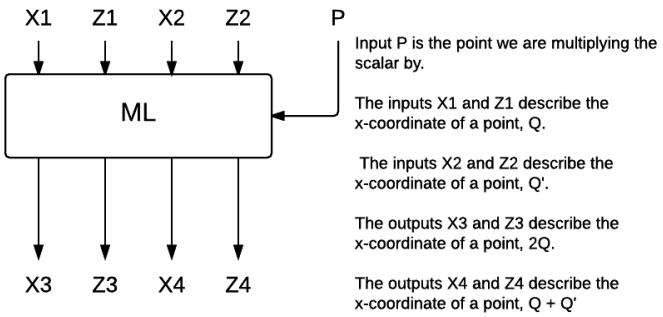


Figure 11: Curve25519 Montgomery ladder

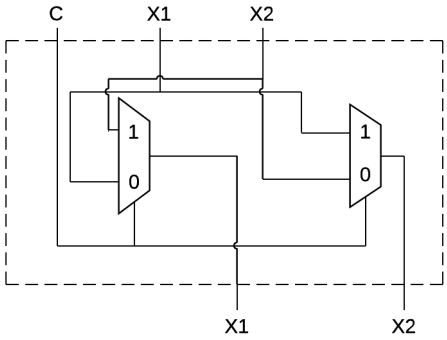


Figure 12: CSwap hardware design

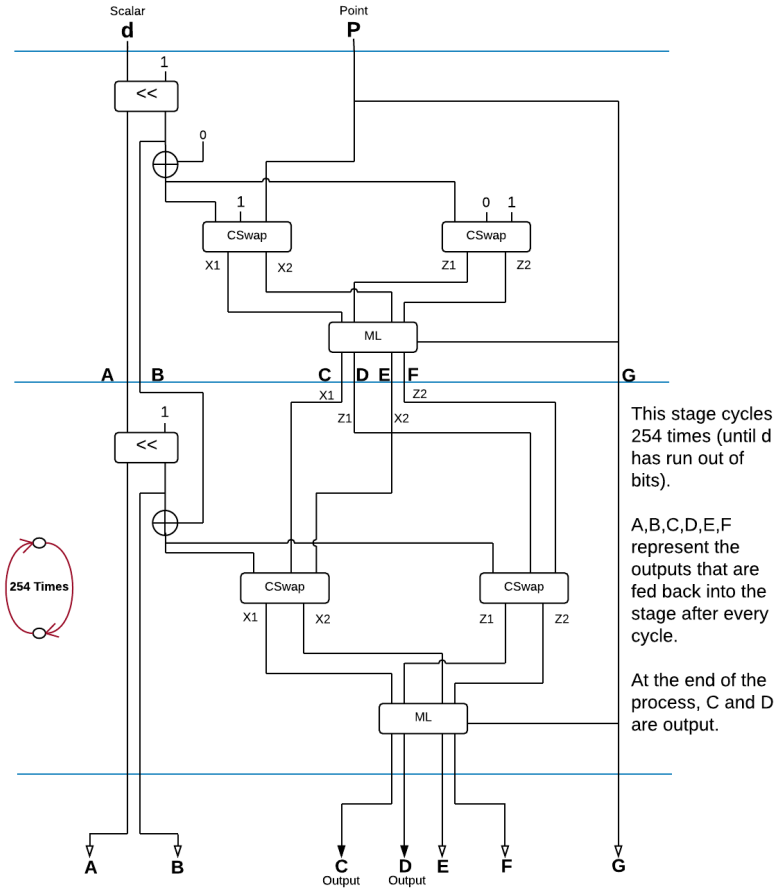


Figure 13: Scalar multiplier hardware design

3.2.8 Symmetric Encryption (Salsa20)

Salsa20 is a symmetric encryption algorithm, that is used in NaCl as an extremely fast primitive for building more advanced cryptographic constructs. Therefore, this design does not contain just Salsa20, but has the component decomposed in various ways to ensure the rest of the system can be modular and reuse primitives. These functional units are more or less modeled after the function names in the original Salsa20 paper [4].

The Salsa20 algorithm encrypts a message divided into 32 bit blocks using a 256 bit key and a 64 bit nonce. Figure 14 contains the detailed design for the core eighth-round and quarter rounds for Salsa20. Salsa20 contains 20 rounds. 10 rounds are "row" operations (4 parallel quarter rounds operating on the different rows of the state matrix) and 10 rounds are "column" operations (4 parallel quarter rounds operating on the different columns of the state matrix).

In hardware, this turns into four quarter-round units being synthesized in parallel, then re-parameterized every round. The full design of a double round may be found in Figure 15. Of course, this core is purely linear, and therefore trivial to invert. This is fixed with HSalsa (Figure 17), which adds the original state to the final state to ensure non-linearity.

In the crypto_box NaCl construction, keys that are 16-bytes and 32-bytes need to be expanded into the full key size required by Salsa20: 64 bytes. Doing this securely is non-obvious, and implemented in Figure 16, which contains designs for expanding subkeys that are both 16 and 32 bytes in length.

Finally, XSalsa is used in crypto_box to extend a key by creating subkeys that are used to encrypt instead. This is detailed in Figure 20. It has its own subkey, whose derivation may be found in the design in Figure 19.

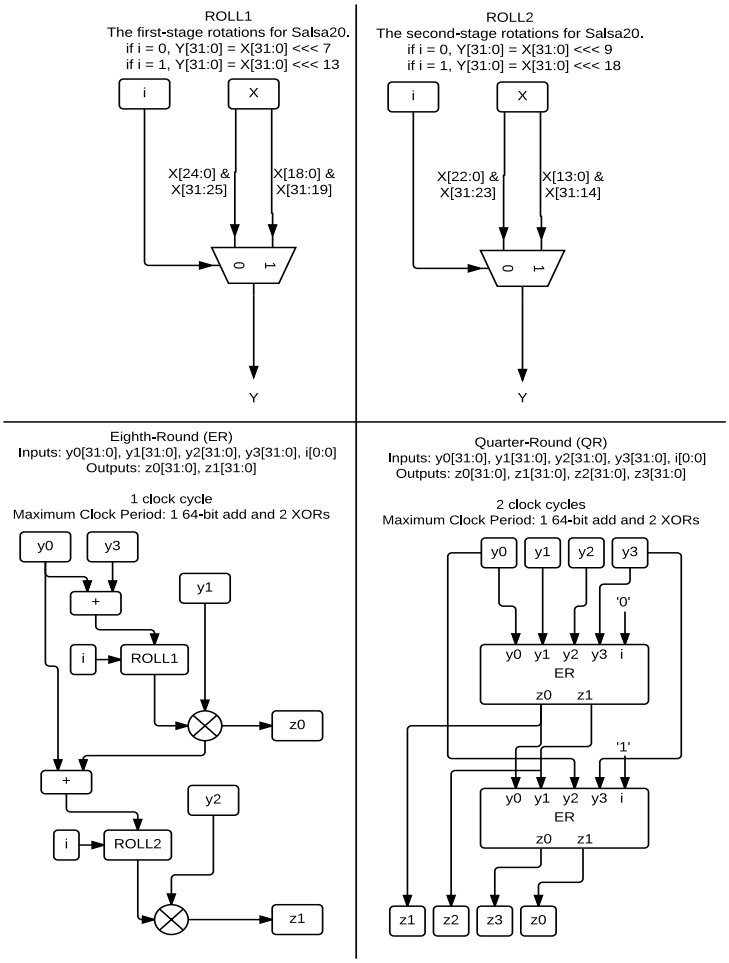
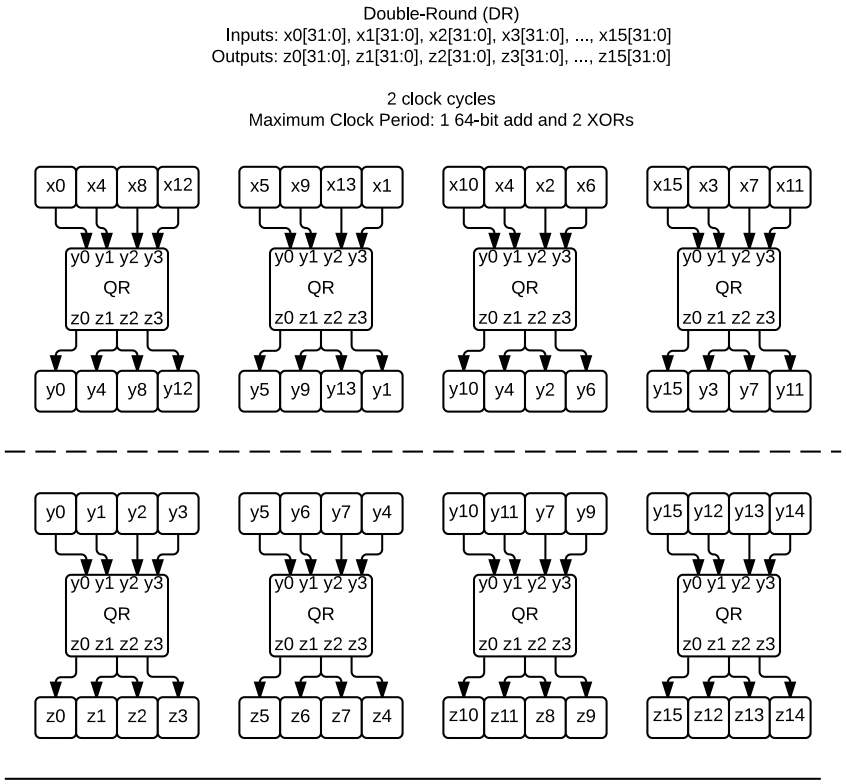


Figure 14: Salsa20 eighth-round and quarter-rounds.



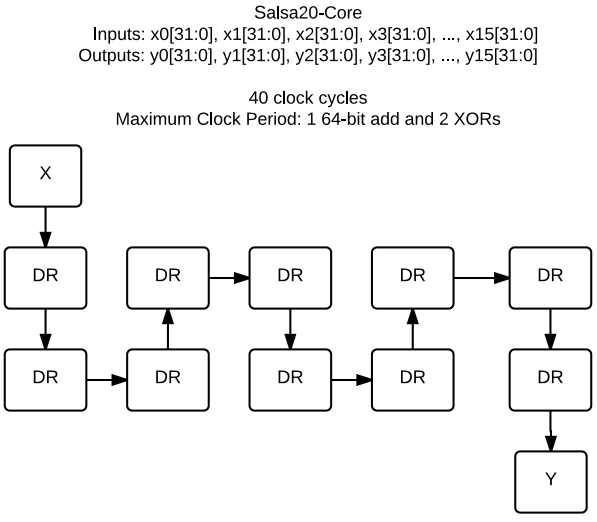


Figure 15: Salsa20 double-round.

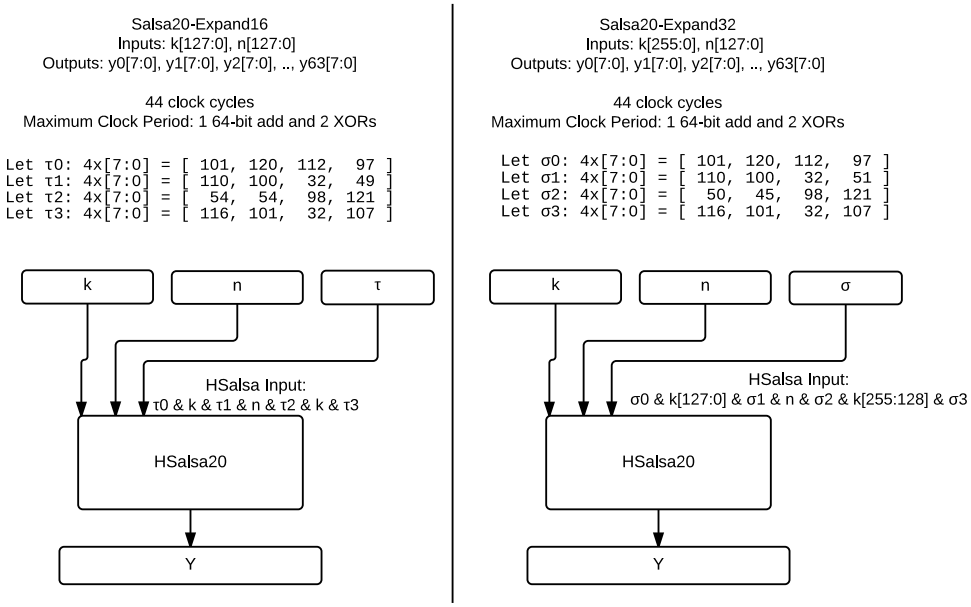


Figure 16: Salsa20 key expansion.

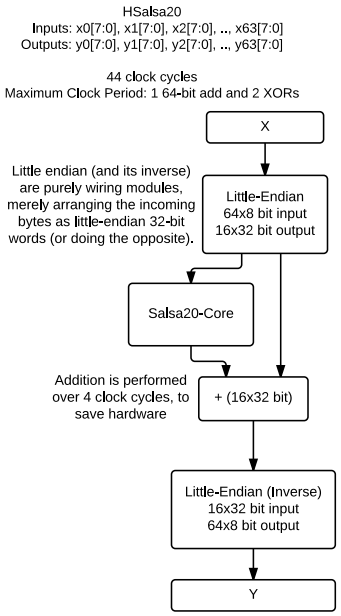


Figure 17: HSalsa20.

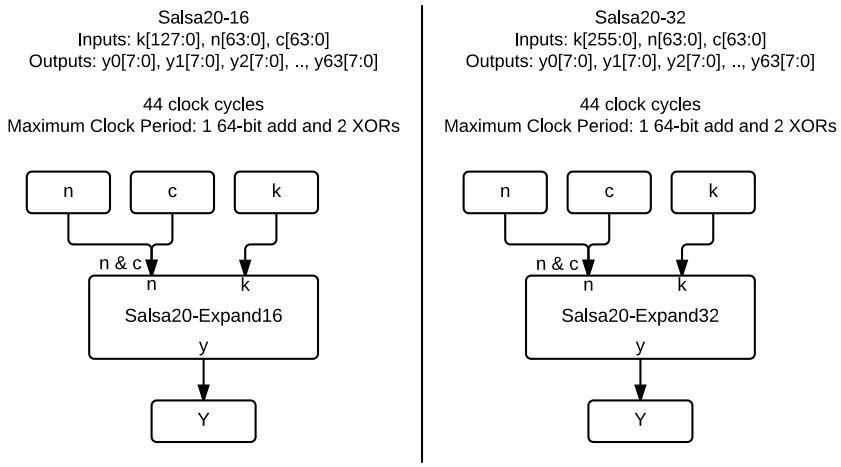


Figure 18: Salsa20.

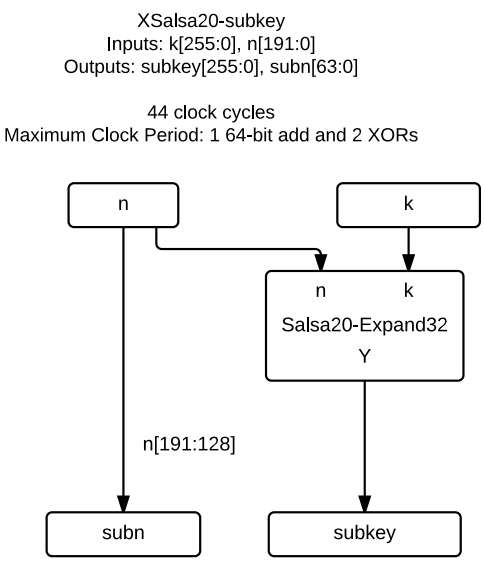


Figure 19: XSalsa20 Subkey.

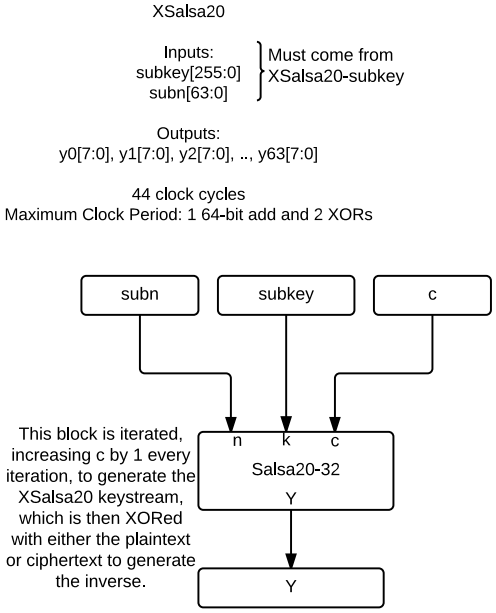


Figure 20: XSalsa20.

3.2.9 Message Authentication (Poly1305)

The Poly1305-Salsa20 message authentication block takes as input a message and a 256 bit Salsa20 secret key, and outputs a 128 bit authentication tag that can be used by the receiver to mathematically prove the integrity and authenticity of the message to dispel man-in-the-middle attacks. The Poly1305 algorithm requires dividing the message into 16 byte blocks and performing accumulating cycles of repeated multiplication by the Salsa20 key and reduction modulo $2^{130} - 5$. The dataflow diagram in Figure 21 details the hardware design.

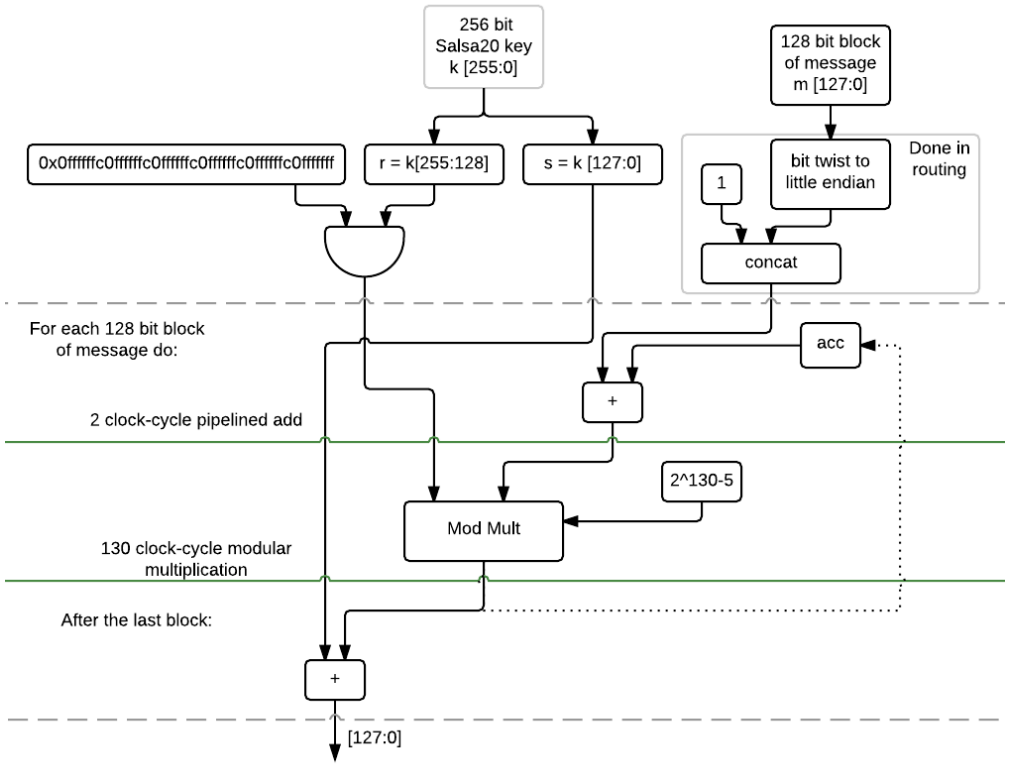


Figure 21: A dataflow diagram of the Poly1305 authentication block

The adder used by the accumulator takes 130 bit operands. As detailed previously, measurements of single clock cycle additions of this size had maximum clock frequencies of 120 MHz. Splitting this addition into two additions of half the size over two clock cycles using a pipelined adder increased the measured frequency to 304 MHz. The reason for this large increase in speed is that the addition must be performed sequentially bit-by-bit in order to calculate and propagate the carry bits. With operands half the size, only half the number of additions and carries are required. This more than doubles the frequency, so even though the latency in terms of clock cycles is greater, it produces results faster and is no longer the bottleneck of the whole system. Additionally, using the 65 bit pipelined adder uses half the area over the full 130 bit adder. It would appear that two 65 adders are required, one for use in each cycle, however advanced design allows the same 65 bit adder can be reused in both cycles of the addition by muxing the inputs. The pipelined adder design is shown in the dataflow diagram of Figure 22.

The modular multiplication is performed over 130 clock cycles, using the functional unit described previously, with Poly1305 specific inputs. The final addition to produce the output is again an addition of 130 bit operands. However the result is reduced modulo 2^{128}. To save clock cycles and reuse hardware, this

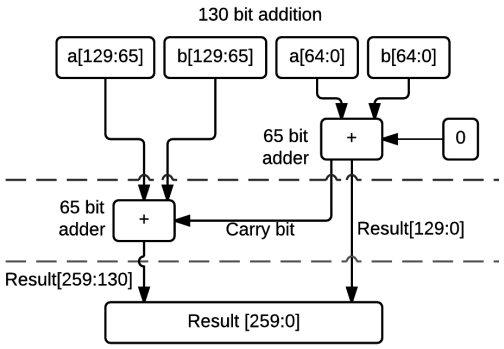


Figure 22: A dataflow diagram of a pipelined 65 bit adder to perform 130 bit addition

addition is optimized as it is only necessary to produce the correct result in the lower 128 bits. This allows the reuse of the same 65 bit adder used previously, with the upper 65 bits of the operands being thrown away as unnecessary.

To further save on clock cycles and area, the initial setup and clamping of the inputs is done in routing and by tying lines to ground, requiring no additional logic. Analysis of the dataflow diagram shows a latency of 133 clock cycles per 16 byte message block. The datapath hardware usage is 642 bit registers, one 65 bit adder and one 130 bit modular multiplier. The fastest, most optimized known Poly1305 software implementations take 308 Pentium 4 cycles to authenticate 16 bytes of a message [3]. The Cortex-M0 used in embedded devices, to which this hardware implementation is being compared, is much less powerful than a Pentium 4 and thus would require at least that many cycles. The hardware design described here meets the specification of being faster than a Cortex-M0 as it takes fewer than half the number or cycles, and at a clock rate higher than the M0's 50MHz. The critical path in performing the authentication is the SPI interface between the microprocessor and the FPGA, which operates at a much lower rate, as described in the following section.

3.3 SPI Interface

The SPI interface is an extremely common chip interconnect standard. It was chosen because of its ease of implementation and high data rates (compared to other common serial interfacing standards such as UART and I2C). A high level diagram of the SPI master is shown in Figure 23. The lines are defined as follows:

- CLK: The master CLK to synchronize data transmission.
- MOSI: Master input slave output. This is a data line.
- MISO: Master input slave output. This is a data line.
- $\overline{SS}$: Slave select. When this line is pulled low, the slave knows to receive transmission
- DATA_READY: This is not part of the SPI interface standard. This is connected to an interrupt pin on the microcontroller to let it know that the module has data ready to be transmitted over the SPI interface. The microcontroller will then assert $\overline{SS}$ and start the clock.

In order to store the data received over the interface, the memory module included on the FPGA board is used. The next two sections will describe the input and output blocks on the FPGA side for receiving and transmitting data.

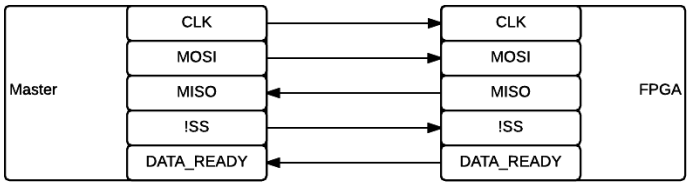


Figure 23: An overview of the SPI interface from the the outside

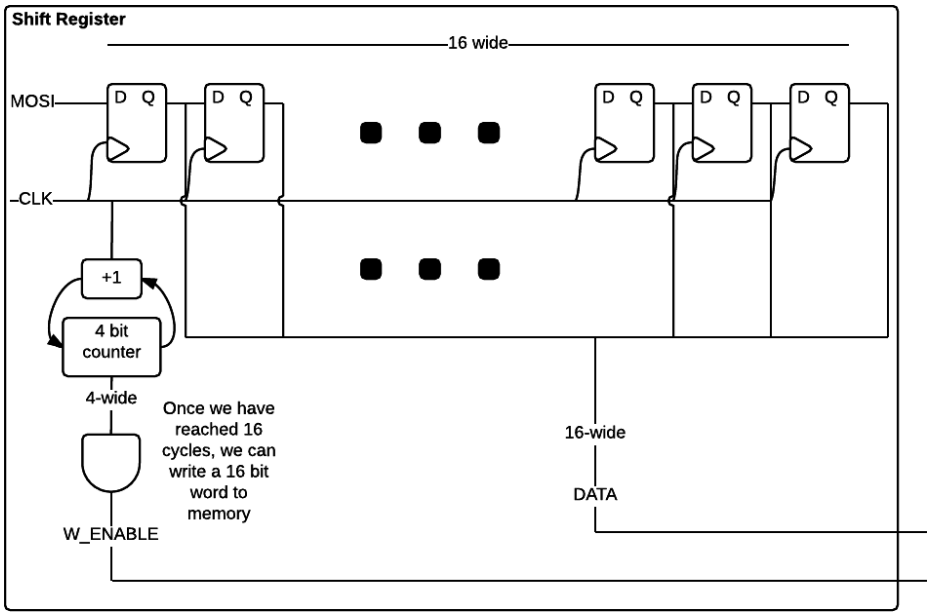


Figure 24: Input Shift register of the SPI interface

3.3.1 Receiving Data

As detailed in Section 3.3 the MOSI line is a serial data line. The memory on the FPGA however is written to 16 bits at a time. Therefore the serial data needs to be converted to parallel data. This is done using a shift register, the diagram for which is shown in Figure 24.

After the data is in the correct format, the next part of the process is writing the data to memory. The memory clock speed can go up to 143 MHz which is much faster than the SPI BUS. Therefore the scenario of receiving data too fast for the memory to be written to is avoided. The dataflow diagram of the transfer from the shift register to the memory is shown in Figure 25. The messages must be padded with 0's for the first 32 bytes for Salsa20, and this padding is done in this stage. The other option was to pad the message with 0's in software on the microcontroller before sending it down the SPI bus. It was decided that doing so would be a waste of time. Transmitting a word over the SPI interface running at its maximum frequency of 5Mhz takes $(\frac{5*10^6 bits/s}{16bits})^{-1} = 3.2 * 10^{-6} seconds$. Running the memory at the speed of the 50 MHz FPGA system clock, 32 bytes of zeroes can be written within the time that the first word is transmitted. Every clock cycle, 16 bits can be written to the memory, meaning it would take 16 clock cycles to write 32 bytes. The amount of time to write 32 bytes of zeros to the memory is as follows: $(\frac{5*10^6 cyckes/s}{16cycles})^{-1} = 3.2 * 10^{-7} seconds$ Therefore, with this solution, a small increase in hardware complexity yields no wasted cycles as well as no

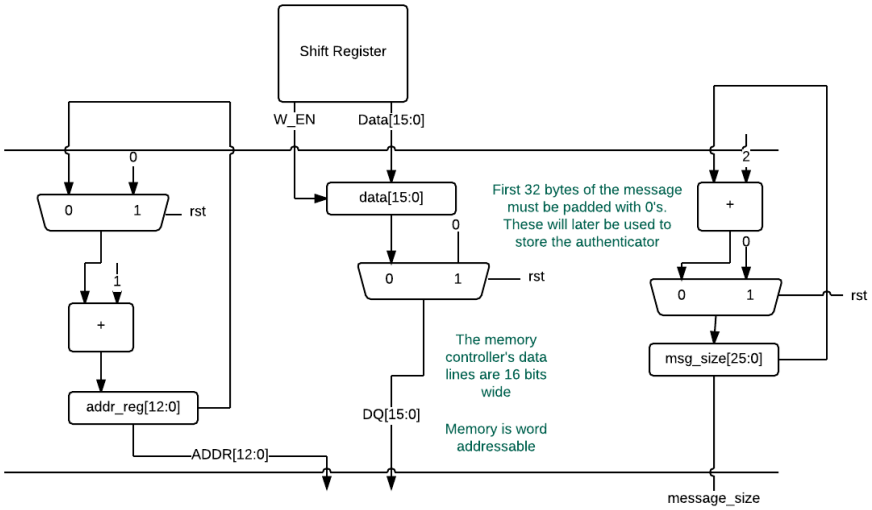


Figure 25: Writing to memory from the shift register

power overhead to send 32 bytes of zeros over the SPI bus.

3.3.2 Sending Data

As is the case in Section 3.3.1 the output to the SPI interface is serial, however the memory will provide 16-bit words at a time. Therefore a parallel to serial conversion must be done. This is also done with a shift register however the overall design must change. Unlike the input stage where the data could be read in from the outputs of the DQ flip-flops, the shift register must have to capability to have all of its data written to in one clock cycle as well as the ability to shift in data from the previous flop. As such, a new type of flop was implemented to make the design easier, described in Figure 26. Upon the write-enable signal being asserted, the flip flop will take the D value. Otherwise it will take the P value (P stands for previous). The configuration is shown in 27. The first flop is a simple DQ flip flop because it has no previous input. It can only take input from the 16-bit data line, triggered by the write enable as opposed to the clock like the other flops that are part of a shift register. The counter in the output has a similar functionality to the one in 25 except that it asserts the WEN signal 1 cycle ahead of time. This allows the data to be put into the shift register while the last bit is being transmitted over the SPI interface.

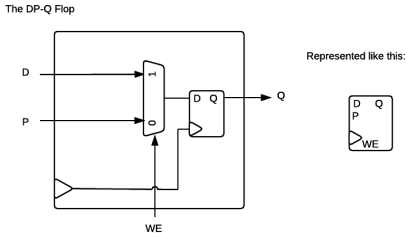


Figure 26: The DP Flip-Flop

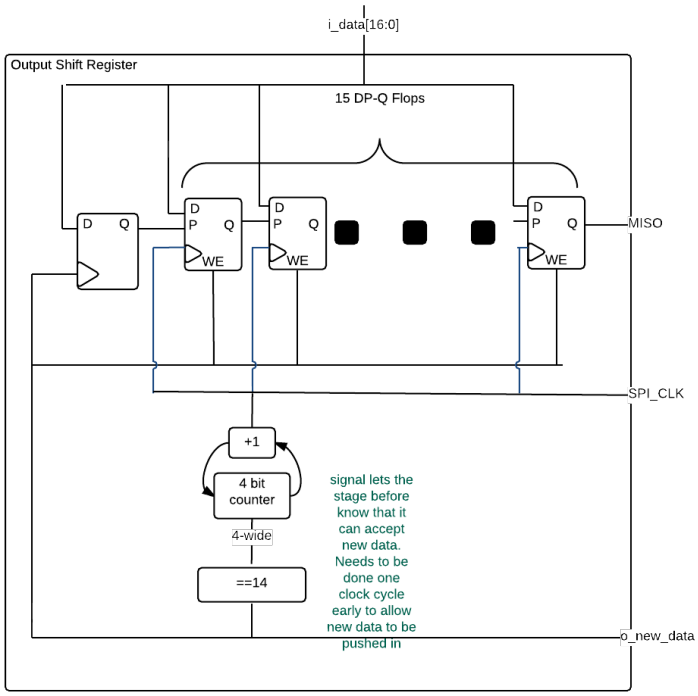


Figure 27: The output shift register

3.4 NaCl-like API

The NaCl API provides a simple front end for the microcontroller's SPI master interface. It starts first creating two control bytes. The first two bytes of the message sent over the SPI bus is used to determine which function the cryptography module will perform. After the two control bytes are created, the message is sent over the SPI bus and the module operates individually. When the module completes operation, it asserts a the *DATA_READY* signal as described in Section 3.3. The user of the API can choose whether or not to poll this signal or to enable an interrupt on that pin.

4 Discussion and Project Timeline

4.1 Evaluation of Final Design

This final design, although quite involved, effectively meets all of the initial objectives and specifications.

There are functional specifications for secure key generation, symmetric encryption, and power consumption. For reference, these are listed in Table 2.

Secure key generation is broken into parts: entropy gathering, noise gathering, noise accumulation, and key storage. These are each designed in detail in Section 3.1.

Symmetric cryptography is "just" a Salsa20 block, which has a detailed design in Section 3.2.8.

Power consumption is a non-essential functional specification, and is measured once the hardware is fully built. In theory, the impact should be much less than doing the cryptography in software. Additionally, the cryptography is performed much faster, with a lower impact on user-visible latency.

Similarly, there are non-functional specs listed in Table 2. They cover physical security, secure key generation, physical interfacing, key generation speed, hardware requirements, and software interfacing.

Keys are stored in tamper-resistant EEPROM, ensuring physical security of the device. Additionally, private keys are never sent off-chip. Therefore, a compromise of the surrounding system will not lead to the compromise of shared secrets. If the higher-level cryptographic protocol uses perfect forward secrecy (which is naturally supported by this designed hardware), even if keys are compromised past communications are not vulnerable. The design for secure key storage is detailed in Section 3.1.

The SPI bus will run at maximum speed. During encryption and authentication, it is easily the bottleneck. Of course, one could choose to run it slower if the trade-offs are deemed appropriate. The design for the SPI physical interface is detailed in Section 3.3.

In terms of hardware requirements, it is as yet unclear if it will fit on the Altera Cyclone IV FPGA. It is conjectured that it will, simply because a great deal of optimization efforts for the particularly large functional blocks has been undertaken. There is also a great deal of single-cycle access RAM on these devices, and is used liberally to reduce space usage, especially for lookup tables. Other than those, the largest elements are by far 256-bit registers. If these can be minimized, possibly by using RAM, the design will have an easier time fitting on the FPGA.

And finally, everything comes together in Section 3.4, which contains an explanation of the C API to the hardware module. This API provides access to the crypto_box primitive[18], which can be used as a foundational building block of secure embedded cryptosystems. The API performs no cryptographic computation of its own, so has a very small RAM footprint on real devices. It just performs interfacing.

4.2 Use of Advanced Knowledge

This project uses many principles of computer security, the theory and algorithms of which are discussed in ECE458, Computer Security, along with different types of attack vectors security systems must defend against. The motivation for the project comes directly from ECE455, Embedded Software, where one of the topics was the lack of security in embedded devices. Skills from ECE455 were also used in designing the SPI interface between the microcontroller and the FPGA, and the API. Extensive knowledge from ECE327, Digital Hardware Systems, was required for designing efficient hardware dataflow diagrams for the cryptographic components, and to synthesize the dataflows in Verilog, which also made use of skills from ECE429. The analog circuitry components used knowledge from ECE242 in which amplifier design was studied. A knowledge of semiconductor materials was also necessary from ECE331 to choose and understand an appropriate noise source.

4.3 Creativity, Novelty, Elegance

A particularly elegant and creative part of this design the elliptic curve point multiplication module. The problem was recursively decomposed into smaller problems: point multiplication to modular operations, and every modular operation decomposes into modular additions and control. In its current form, elliptic curve point multiplication is a few parallel 64-bit adders with a significant amount of control circuitry. This is of great help in keeping clock period under control, since this is by far the biggest component in the design. Additionally, many subcomponents have variable amounts of parallelism, allowing tuning of area/time tradeoffs.

Another part of this design that is particularly elegant is the NaCl crypto_box API in general. Although not designed by the authors of this report, the choice to use it has led to very understandable and defensible cryptographic hardware. There are are many defense in depth aspects to the design, although it's based off of only a few, simple cryptographic primitives. It was also designed to be resistant to timing attacks, which is especially convenient for hardware design. This means that all cryptographic operations take a fixed amount of time, and can be statically scheduled across clock cycles.

In the course of designing the hardware for this device, there were some components whose correctness was dubious. In order to catch design errors before synthesis, these components were formally verified with an SMT solver. Specifically, the modular arithmetic components were verified to be equivalent to their simple (but slow) solutions. This was done with the assistance of the Z3 SMT solver, and the SBV Haskell library.

4.4 Student Hours

Table 4: Total Student Hours Worked

Student	Hours
[redacted]	107
[redacted]	70
[redacted]	67
[redacted]	69
[redacted]	68

Table 4 summarizes the total number of hours worked so far on this project.

4.5 Potential Safety Hazards

The primary safety hazard relates to the use of custom circuitry for the key generator and the SPI interface. The circuitry uses low voltages and currents, so the risk of harm due to contact is low, however the system will need to be enclosed in a plastic box. This will protect against exposure of live wires or voltage rails, as well as against any sharp edges such exposed pins of the FPGA or the microcontroller.

PCBs have sharp corners, and can be a safety hazard. Care must be taken during prototype construction. As a rule of thumb, no unnecessary touching of the PCB is allowed.

4.6 Project Timeline

Given the nature of hardware design, many of the components can be designed in parallel. For example, as far as the designer of the Montgomery ladder is concerned, the modular multiplier is a black box that will exist at some point. However, verification cannot occur until all components are built. The RTL coding and API building represent the prototype construction of the design. During these phases, the hardware is being designed and by the end there should be a working prototype running on the FPGA. For the symposium preparation phase, the prototype is being made neatly presentable and safe for the public.

The RTL coding phase has already started and the project is well on track to being at least 50% finished in time for the prototype demonstration. A timeline of the project is presented in Figure 28. Some time for design revision has been allocated during the RTL coding stage to allow for iterations on the design due to new challenges that only become apparent once implementation has begun.

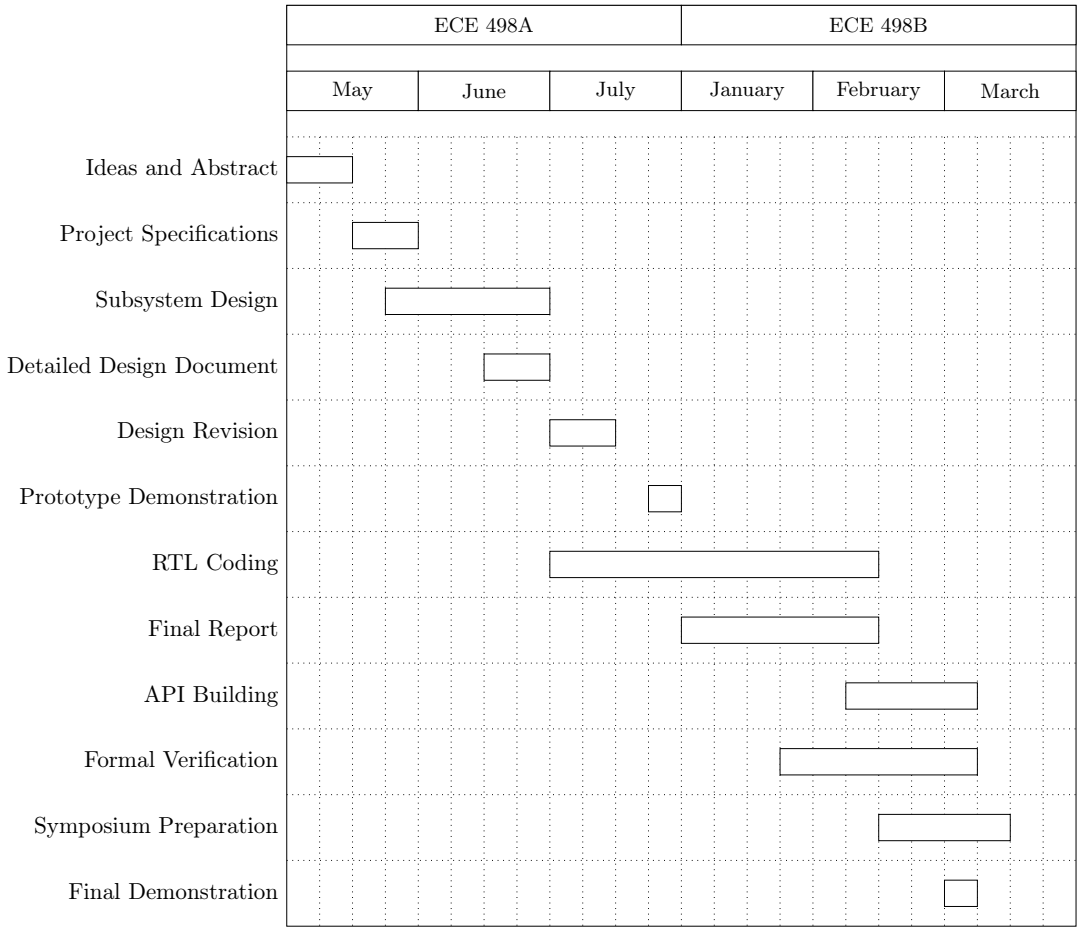


Figure 28: Project timeline

References

[1] Hewlett Packard, "*Internet of Things Research Study*," 2014. [Online]. Available: http://h20195.www2.hp.com/V2/GetDocument.aspx?docname=4AA5-459ENW&cc=us&lc=en

[2] M. Zhang, A. Raghunathan, and N. Jha, "Trustworthiness of Medical Devices and Body Area Networks," *Proceedings of the IEEE*, vol. 102, no. 8, pp. 1174–1188, Aug. 2014.

[3] D. J. Bernstein, "The Poly1305-AES Message Authentication Code," *Fast Software Encryption*, vol. 12, no. 12, pp. 42–39, Mar. 2005. [Online]. Available: http://cr.yp.to/mac/poly1305-20050329.pdf

[4] D. J. Bernstein, "The Salsa20 Family of Stream Ciphers," *New stream cipher designs: the eSTREAM finalists*, Dec. 2007. [Online]. Available: http://cr.yp.to/snuffle/salsafamily-20071225.pdf

[5] D. J. Bernstein, "Curve25519: New Diffie-Hellman Speed Records," *Public key Cryptography*, no. 9, Apr. 2006, http://cr.yp.to/ecdh/curve25519-20060209.pdf.

[6] D. J. Bernstein, "Cryptography in NaCl," *Journal of Cryptology*, Mar. 2009. [Online]. Available: http://cr.yp.to/highspeed/naclcrypto-20090310.pdf

[7] "A. Rukhin, J. Soto, et. al.", "A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications," NIST Standard, NIST, Special Publication 800-22, apr 2010. [Online]. Available: http://csrc.nist.gov/groups/ST/toolkit/rng/documents/SP800-22rev1a.pdf

[8] N. Mouha, B. Mennink, et. al., "Chaskey: An Efficient MAC Algorithm for 32-bit Microcontrollers," *Selected Areas in Cryptography*, 2014. [Online]. Available: https://eprint.iacr.org/2014/386.pdf

[9] D. J. Bernstein, *Why switch from AES to a new stream cipher?*, 2005. [Online]. Available: http://cr.yp.to/streamciphers/why.html

[10] L. Batina and M. Robshaw, *Cryptographic Hardware and Embedded Systems – CHES 2014: 16th International Workshop, Busan, South Korea, September 23-26, 2014, Proceedings*, ser. Lecture Notes in Computer Science / Security and Cryptology. Springer Berlin Heidelberg, 2014.

[11] Maxim Integrated, "Building a Low-Cost White-Noise Generator," 2004. [Online]. Available: http://www.maximintegrated.com/en/app-notes/index.mvp/id/3469

[12] D.J. Bernstein, "What output size resists collisions in a xor of independent expansions?" *Workshop Record of ECRYPT Workshop on Hash Functions*, Mar. 2007. [Online]. Available: http://cr.yp.to/rumba20/expandxor-20070503.pdf

[13] D. N. Amanaor, "Efficient Hardware Architectures Modular Multiplication," Ph.D. dissertation, University of Applied Sciences, Offenburg, Germany, Feb. 2005.

[14] T. Koshy, *Elementary number theory with applications.* Amsterdam Boston: Academic Press, 2007.

[15] B. Schneier, *Applied cryptography : protocols, algorithms, and source code in C.* New York: Wiley, 1996.

[16] Yann Loisel, "Cryptography in software or hardware: It depends on the need," *International Association for Cryptologic Research*, 2015. [Online]. Available: https://eprint.iacr.org/2015/343.pdf

[17] Michael Düll, Björn Hulle, Gesine Hinterwälder, Michael Hutter, Christof Paar, Ana Helena Sánchez, Peter Shwabe, "High-speed Curve25519 on 8-bit, 16-bit and 32-bit microcontrollers," *Embedded (UBM Electronics)*, 2011. [Online]. Available: http://www.embedded.com/design/mcus-processors-and-socs/4219372/Cryptography-in-software-or-hardware--It-depends-on-the-need

[18] D.J. Bernstein, "Public-key authenticated encryption: crypto_box," 2010. [Online]. Available: http://nacl.cr.yp.to/box.html